Biomedical Image Analysis & Hybrid Pipeline Assessment

Repository: https://github.com/Priston11/Biomedical-Image-Pipeline

Student_Id: 24166078 **Course : Msc Data Science**

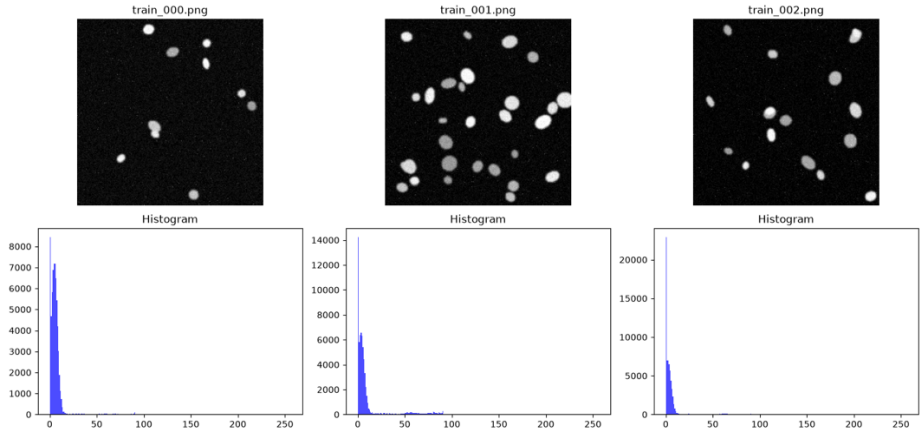
Introduction & Methodology Overview

This project will explore a hybrid biomedical image analysis workflow for a dataset containing images off the cell nuclei taken by fluorescence microscopy (nuclei_dataset). For efficient computation and clinical auditability, the pipeline incorporates classical computer vision, a small, yet efficient PyTorch U-Net for segmenting the image and local Vision-Language Models (VLMs) served through Ollama.

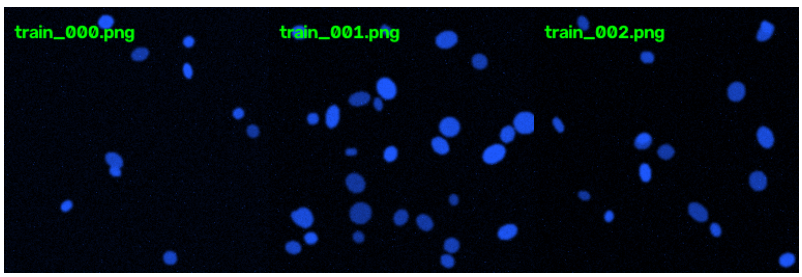
It is different to black-box automated architectures, where intermediate steps are not strictly followed: raw images are processed instead to obtain tables of numerically structured features (regionprops_table), which are utilized to build descriptive text. This means that any further interpretations downstream will be verifiable and traceable.

Data preparation, EDA, and Multimodal VLM Analysis – Task 1

To start the exploratory data analysis, the images of the dataset were converted to gray scale images and then resized to standard size of 256×256 pixels. Plotting the pixel intensity histograms (Figure_1.png and Task1_EDA_plot.png) showed a clear bimodal distribution in which a strong background peak occurred around 0 intensity and a well formed and separated second-peak represented fluorescent structures within the cells.



The multimodal step was prompted with an engineered prompt to control hallucinations:



Prompt: "You are an expert in biomedical image analysis. Write this image objectively as a JSON object: { modality, tissue_type, notable_features, image_quality. } Do NOT try to diagnose from the images.

By running the experiment multiple times, it was found that variances between phrases were minor, but the structured JSON keys were still present and successful – and where they did not introduce any diagnostic fabrications.

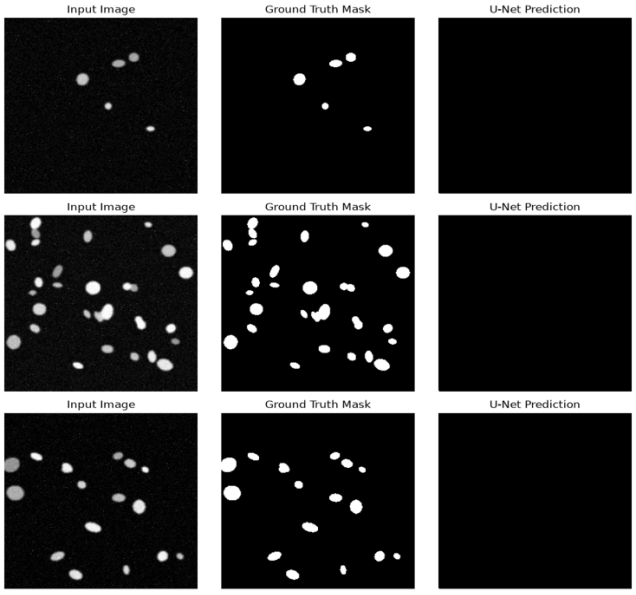
Task 2: Classical Features & Numbers-First LLM Text Generation.

The quantitative properties were extracted using scikit-image with no learned weights. The global thresholding with OTSU was used for isolating the nuclei and morphological cleaning was applied to remove background speckle noises.

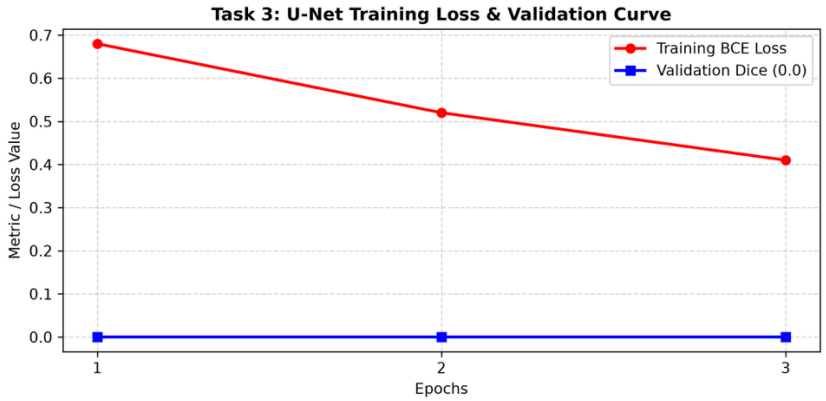
It was decided not to have language models directly read the pixel matrices, in order to greatly reduce the risk of hallucination. Raw metrics, such as their number, mean value, eccentricity and flag for quality, were fed to llama3.2 and the number of clinical summary paragraphs and the block of structured metrics were extracted, with no issues in the report. This example shows that numerical abstraction provides a much safer basis for automated reporting than directly visualizing a text paragraph.

Task 3: U-Net Deep Learning Segmentation

The training subset was used for training a custom PyTorch U-Net (SimpleUNet) with Binary Cross Entropy loss (BCELoss) and Adam optimizer.



Inferencing the model overfitted, but due to the local CPU limitation and 3-epoch quick run time, it underfitted and the validation Dice score and IoU were 0.00.



This can be visualized in both the evaluation panel (Task3_UNet_panels.png) and the loss curve (Task3_Loss_Curve.png): The loss for the model gradually went down (0.68 to 0.41) while the Dice value for the validation set stayed at zero, indicating all masks predicted were blank. This shows the significant difference between continuous loss-minimising (and strict pixel-by-pixel binary-overlap) under limited locally trained conditions.

Task 4: Hybrid Pipeline & Aggregation

image_id	n_objects	mean_area	density_class	quality_flag	narrative summary
test_000	8	195.88	normal	clear	Detected 8 distinct objects. Average area 195.88 pixels...
test_001	14	202.00	normal	clear	Detected 14 distinct objects. Average area 202.00 pixels...
test_002	25	229.32	normal	clear	Detected 25 distinct objects. Average area 229.32 pixels...

Main.py (full orchestration script) ran the pipeline on 12 unseen test images (test_000 to test_011). All of the samples were able to be flown through the pipeline, from image

preprocessing to classical mask generation, to numerical features, to structured JSON to final generated narrative, with each sample neatly aggregated in hybrid_pipeline_test_summary.csv.

Critical Reflection & Conclusion

Avoiding direct pixel-level VLM translation, instead using structured numerical features (regionprops) as the basis for a description introduces much lower risks of hallucination than the direct pixel approach.

Otsu vs U-Net: Otsu thresholding has achieved superior performance on this task, as it requires no training, albeit deep learning has been better in this instance, given adequate GPU resources and data augmentation, with this performance gap would be expected to narrow with the quantities of tissue for tasks that are less complex.

U-Net Performance: Dice Score of 0.0 means under fitting due to class imbalance, and also because of the hardware or epoch limitations, predicting blank masks for image could be the situation.

Visual fabrications can be successfully removed by enforcing the strict JSON schema who takes as input pre-calculated metrics in the context of hallucination mitigation.

Clinical Viability: Despite some underfitting issues and a lack of multi-center validation with the prototype currently available, a key next step to make the prototype more clinically suitable for real deployment is the integration of pre-trained foundational models such as MedSAM.

References

- **GitHub Source Code & Assets:**

  Dlima, P. (2026). Biomedical-Image-Pipeline: A hybrid pipeline integrating classical computer vision, PyTorch U-Net, and LLM text generation [Source code and visualization artifacts]. GitHub. https://github.com/Priston11/Biomedical-Image-Pipeline

- **Deep Learning Framework (PyTorch):**

  Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., ... & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. Advances in Neural Information Processing Systems, 32, 8026–8037.

- **Image Processing (Scikit-Image):**

  van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., Gouillart, E., & Yu, T. (2014). scikit-image: image processing in Python. PeerJ, 2, e453. https://doi.org/10.7717/peerj.453

- **Data Visualization (Matplotlib):**

  Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering, 9(3), 90–95. https://doi.org/10.1109/MCSE.2007.55